

Duração: 1h00. Tolerância: 15
minutos

Sem consulta

Forma da resposta - 3 folhas de resposta - 1 pergunta por folha

- Todas as folhas devem estar identificadas com o **nome completo e número de aluno** em caligrafia legível.
- Deve entregar as **3 folhas de resposta** mesmo aquelas que estiverem **vazias** (por responder).

1. [30%] Implemente uma classe **Temp** que representa medições de temperatura. Uma medição de temperatura é definida pelo local da medição (uma cadeia de caracteres de tamanho indefinido), a temperatura, em Celsius (um número não-inteiro), e a altitude em metros (um número não-inteiro).

Nenhum objeto desta classe deverá poder existir sem a especificação de todos os seus dados. Todos os dados dos objetos desta classe devem poder ser obtidos. O valor da temperatura pode ser modificado; o local e a altitude não.

O valor da temperatura deve ser igual ou superior ao valor do zero absoluto que é -273.15. O valor do zero absoluto deve ser representado por um dado membro que seja partilhado por todos os objetos da classe (não faz sentido estar a repetir este valor, pois é sempre o mesmo para todos os objetos). Deve haver um método para atualizar o valor do zero absoluto (para o caso de as leis da física mudarem) e esse método deve poder ser chamado mesmo que não haja objetos nenhuns da classe. Caso este valor seja mudado, todos os novos objetos, assim como as atualizações dos que já existirem, passarão a cumprir esse novo valor mínimo.

Deve ser possível obter uma representação textual com os dados do objeto.

Todas as funções que não alteram o objeto devem poder ser chamadas sobre objetos (desta classe) que sejam constantes.

Garanta que não impede o uso dos objetos da classe nas operações habituais normalmente disponíveis para objetos de classes.

O código deve ser apresentado separado em .h e .cpp. Não precisa de especificar #includes da biblioteca C++ – apenas dos seus .h. Não se pretende função *main* nenhuma.

2. [30%] Pretende-se uma classe **Relógio** cujos objetos armazenam as horas e os minutos. Um relógio pode ser inicializado com nada, com um inteiro (horas), ou com dois inteiros (horas e minutos). Os valores válidos são: 0 a 23 (horas) e 0 a 59 (minutos), devendo sendo corrigidos se forem indicados valores inválidos.

Pretende-se que sejam possíveis as seguintes operações:

// a e b são relógios

```
cout << "horas aqui: " << a << " horas ali: " << b;
```

```
if ( a ) { cout << "não são zero horas e zero minutos"; }
```

```
if (a == b) cout << "os relógios têm a mesma hora (horas e minutos)";
```

```
cout << ++a; // acerta o relógio (mais 1 minuto, e talvez hora, e mostra como ficou)
```

```
a = b; // o relógio a fica com as mesmas horas e minutos que b (mantém a marca)
```

```
a += 15; // adianta o relógio 15 minutos (pode assumir que o valor é sempre positivo)
```

```
a[0] = 7; // acerta a hora do relógio a (7 horas e os minutos que já estavam antes)
```

```
b[1] = 30; // acerta os minutos (apenas) do relógio b (? horas e 30 minutos)
```

// Nota: nestes dois últimos casos apenas 0 e 1 serão aceites.

A sua implementação deve garantir que o relógio nunca fica com valores inválidos de horas e minutos exceto os casos onde essa verificação ultrapasse o alcance do seu código.

Não precisa separar o seu código em .h e .cpp. Não precisa de especificar #includes nem namespaces. Não se pretende função *main* nenhuma.

3. [40%] Pretende-se representar uma orquestra de músicos. Há dois conceitos principais (sem exclusão de outros), que são os músicos e as orquestras. Um músico é definido pelo nome do instrumento que toca, pelo seu nome, e pela sua experiência. O nome do instrumento e o nome do músico são cadeias de caracteres de tamanho indeterminado; a experiência é um inteiro. O nome do instrumento, se nada for dito, pode ser "NENHUM", mas nenhum músico deve existir sem o seu nome. A experiência (inicial) é 0.

Nota: o conceito de músico não representa uma pessoa: trata-se apenas de uma espécie de "ficha de informação" que existe apenas no contexto de uma determinada orquestra e apenas aí.

Quanto à orquestra:

- Uma orquestra tem um nome e até 20 músicos. A sua criação exige o nome, e qualquer número de músicos, desde que não se ultrapasse o número máximo de músicos da orquestra.
- Deve ser possível acrescentar um músico à orquestra dadas as informações do músico.
- Deve ser possível remover um músico dado o seu nome e o nome do instrumento (se houver vários, remove o primeiro que encontrar).
- Deve existir um mecanismo que simula a orquestra dar um concerto. Sempre que isso acontece, a experiência dos músicos é incrementada, exceto para aqueles que tocam instrumentos chamados "NENHUM".

Importante:

- Esta pergunta **não envolve memória dinâmica** e **não deve usar matéria que não foi ainda dada**.
- Caso ache que precisa de **modificar algum dos requisitos relativamente ao músico** face aos que foram dados no início do enunciado, **pode** fazê-lo desde que o **justifique** e que essa justificação **faça sentido**.

Implemente o necessário para ter orquestras com músicos.

Não precisa separar o seu código em .h e .cpp. Não precisa de especificar `#includes` nem `namespaces`. Não se pretende função `main` nenhuma.